

Introduction à la POO et à l'héritage avec RoboCode

Robocode

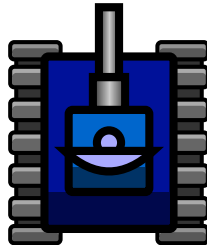


Figure 1: Logo RoboCode

Robocode est un jeu de programmation multijoueur dont l'objectif est de vous faire développer un robot (bot) qui pilote un tank et combat les autres joueurs.

Les jeux de programmation sont un excellent moyen d'apprendre à coder en s'amusant. Voici quelques autres titres connus du genre :

- Human Resource Machine
- The Farmer Was Replaced : <https://thefarmerwasreplaced.com/terms/>
- CodeCombat : <https://codecombat.com/>

Liens utiles

- Site officiel de Robocode : <https://robocode.dev>
- Tutoriel de mise en place d'un bot : <https://robocode.dev>
- Anatomie d'un bot : <https://robocode.dev/articles/anatomy.html>
- Système de coordonnées : <https://robocode.dev/articles/coordinates-and-angles.html>
- Physique : <https://robocode.dev/articles/physics.html>
- Gestion des collisions : <https://robocode.dev/articles/collision-mechanics.html>
- Score : <https://robocode.dev/articles/scoring.html>

Le bot de base, héritier de la classe Bot

RoboCode fournit des scripts de bots ayant différents comportements. Vous pouvez les récupérer en utilisant l'archive suivante : <https://github.com/robocode-dev/tank-royale/releases/download>

ad/v0.38.1/sample-bots-python-0.38.1.zip et vous en inspirer lors de la création de vos propres bots RoboCode.

Tous les extraits de code fournis dans ce document doivent être testés via le serveur RoboCode présent sur votre machine.

Copiez le code suivant pour obtenir le bot le plus basique possible (un bot qui ne fait rien) :

```
from robocode_tank_royale.bot_api.bot import Bot

class ABotThatDoNothing(Bot):
    pass

def main() -> None:
    bot = ABotThatDoNothing()
    bot.start()

if __name__ == "__main__":
    main()
```

Cet exemple, bien que minimal, permet déjà de comprendre plusieurs choses :

- Un bot doit **hériter** de la classe Bot du package `robocode_tank_royale.bot_api.bot`. Hériter signifie que la classe `ABotThatDoNothing` récupère les caractéristiques nécessaires pour être utilisée comme un bot par le serveur RoboCode.
- Un bot doit être démarré en appelant la **méthode** `start()`. La ligne `bot.start()` est donc indispensable pour que le programme commence à communiquer avec le serveur RoboCode.

Pour rappel, dans RoboCode le script de votre robot peut-être utilisé pour piloter plusieurs robots d'un même match. C'est pour cette raison qu'il est contenu dans une **classe** et chaque char contrôlé sera une **instance** de cette classe. La **classe** Bot est centrale, elle permet à la fois au serveur de vous communiquer des informations sur les statistiques de votre robot ainsi que le match en cours et à votre robot d'effectuer des actions.

Voici un diagramme de classe qui reprend de manière non-exhaustif les différents **attributs** et les différentes **méthodes** de la classe Bot :

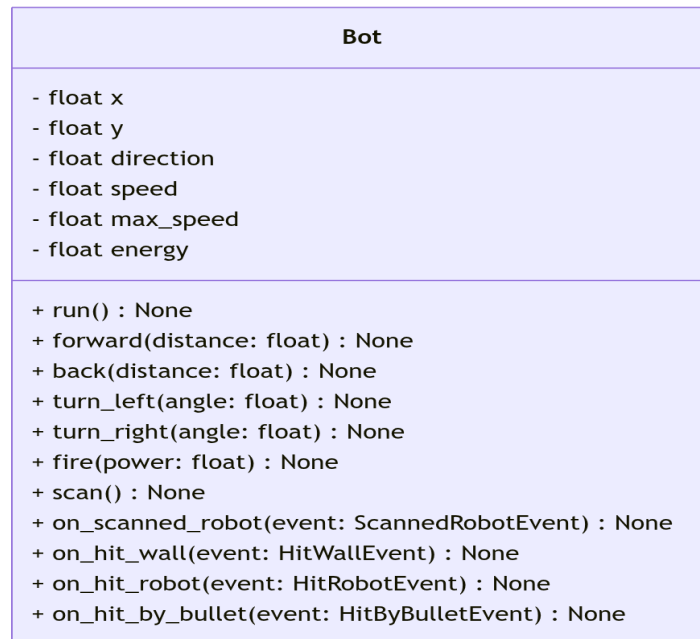


Figure 2: Diagramme de la classe Bot

Pour rappel, les attributs sont à voir comme des variables attachées à **l'instance** (`self`) et servent à **stocker des informations**. Les méthodes, particulièrement dans roboCode, servent à **effectuer des actions** et à **réagir à des évènements**.

Informations :

- x, y : la position du robot dans la zone de jeu
- direction : la direction du char
- speed : la vitesse actuelle du char
- max_speed : la vitesse maximum du char

Effectuer des actions :

- forward(distance) : faire avancer le char
- back(distance) : faire reculer le char
- turnLeft(angle) / turnRight(angle) : tourner d'un angle (en °)
- fire(power) : tirer avec une puissance (entre 0.1 et 3)
- scan() : force un scan radar

Réagir à des évènements :

- on_scanned_bot(event) : appelé quand un ennemi est détecté
- on_hit_wall(event) : quand le char touche un mur

- `on_hit_bot(event)` : lors d'une collision avec un autre char
- `on_hit_by_bullet(event)` : quand le char se fait tirer dessus
- `run()` : appelé lors du démarrage du robot

Déplacer le bot

Afin que notre bot fasse quelque chose de plus intéressant, nous allons le faire se déplacer et tourner son canon.

Fonctionnement tour par tour

Un combat dans Robocode peut comporter plusieurs manches. Par exemple, un combat peut contenir 10 manches distinctes, chacune ayant ses vainqueurs et ses perdants.

Chaque manche est divisée en **Tours** , qui constituent les plus petites unités de temps. Un tour correspond à un "tic" de l'horloge et à une itération de la boucle de jeu.

À chaque tour, un bot doit :

- Se déplacer, rechercher des ennemis et éventuellement tirer
- Réagir à des événements (collision, tirs reçus, etc.)

Les commandes (déplacement, rotation, tir...) sont envoyées au serveur sous forme **d'intentions** pour chaque tour.

Copiez le code suivant afin d'avoir un bot qui avance et tourne son canon :

```
class ABotThatDoNothing(Bot):
    def run(self) -> None:
        while self.running:
            self.forward(100)
            self.turn_gun_left(360)
            self.back(100)
            self.turn_gun_left(360)
```

Ici :

- La méthode `run()` provient de la classe `Bot`
- On dit qu'elle est **redéfinie (override)** dans `ABotThatDoNothing`
- `self.running` est un attribut de la classe `Bot` qui permet, ici aussi, de récupérer des informations nécessaires au bon fonctionnement de notre bot.
- `self.forward` / `self.turn_gun_left` / `self.back` sont des méthodes qui permettent de faire des opérations auprès du serveur (avancer, reculer, tourner la tourelle)

Cela crée un **contrat** entre les classes :

- La classe `Bot` garantit que `run()` sera appelée à chaque début de **manche**
- Votre classe définit le comportement du bot pendant la manche

Votre bot doit fonctionner pendant toute la durée de la manche une boucle `while` conditionné à la valeur de `self.running` est donc nécessaire pour votre bot exécute en boucle ses instructions.

La classe `Bot` a beaucoup d'autres choses à vous offrir pour vous permettre d'obtenir un robot complet, vous pouvez découvrir toutes les possibilités en utilisant cette documentation : https://robocode.dev/api/python/api/bot_api.html#module-bot_api.bot

Des bots spécialisés

Un char dans RoboCode est constitué de trois éléments :

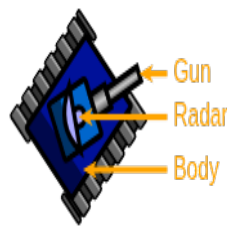


Figure 3: Composants d'un char dans RoboCode

- Le corps (body) : déplacement du tank
- La tourelle (gun) : tir
- Le radar (radar) : détection des ennemis

L'objectif est de créer trois bots spécialisés :

- Un bot qui contrôle le radar
- Un bot qui contrôle la tourelle
- Un bot qui conduit le char

Cela permet de **séparer les responsabilités**.

Pour cette partie, n'hésitez pas à jeter un oeil à la documentation de la classe `Bot` : https://robocode.dev/api/python/api/bot_api.html#bot_api.base_bot

RadarOperatorBot : le bot qui contrôle le radar

```
class RadarOperatorBot(Bot):  
    def run(self) -> None:  
        while self.running:  
            self.turn_radar_left(360)
```

Le rôle de ce bot est de scanner en permanence le champ de bataille afin de détecter les ennemis.

Objectifs

1. Faire tourner le radar en continu pour balayer toute la zone.
2. Modifier le comportement pour que le radar tourne sans interruption.
3. En vous appuyant sur la documentation, implémentez une réaction (stopper le radar par exemple) lorsqu'un ennemi est détecté.

Indice : essayez de trouver un exemple d'utilisation de la méthode `on_scanned_bot(...)`

TurretOperatorBot : le bot qui contrôle la tourelle

```
class TurretOperatorBot(Bot):  
    def run(self) -> None:  
        while self.running:  
            self.turn_gun_left(360)  
            self.fire(1)
```

Le rôle de ce bot est de diriger la tourelle vers les ennemis et de tirer tout en évitant la surchauffe du canon.

Objectifs

1. Faire tourner la tourelle et tirer en continu.
2. En vous appuyant sur la documentation : https://robocode.dev/api/python/api/bot_api.html#bot_api.base_bot.BaseBot.gun_heat

Mettre en place un bot qui tire uniquement lorsque le canon n'est pas en surchauffe.

3. Mettre en place un tir à puissance aléatoire tant que le canon n'est pas en surchauffe.
4. Pour les plus avancés : adaptez la puissance de tir en fonction de la distance de l'ennemi.

PilotBot : le bot qui conduit le char

```
class PilotBot(Bot):  
    def run(self) -> None:  
        while self.running:  
            self.forward(100)  
            self.turn_left(90)
```

Le rôle de ce bot est de déplacer le char et d'éviter d'être une cible facile.

Objectifs

1. Implémenter un déplacement simple (avancer, reculer) avec une rotation.
2. Réagir lorsqu'un mur est touché.

Indice : essayez de trouver un exemple d'utilisation de la méthode `on_hit_wall(...)`

3. Mettre en place un mouvement continu vers une destination.
4. Pour les plus avancés : Modifier le comportement lorsqu'un tir est reçu.

Indice : essayez de trouver un exemple d'utilisation de la méthode `on_hit_by_bullet(...)`